

Name: K RAJ BHARATH K RAJ BHARATH | Reg. No: RA2511003011548

Unit – 1 : Session – 2 : SLO – 2 | Array of Structures

■ I. Fill in the Blanks

1. multiple records — an array of structures stores many records of the same type.
2. `arrayName[size] — struct StructName arr[100];`
3. `arr[index].member` — access a member via index and dot operator.
4. 5 Student structures — `struct Student s[5];` declares an array of 5 Student records.
5. student/employee records — e.g. roll numbers, names, salaries.
6. loop (for loop) — read values into the array with a loop and `scanf`.
7. `emp[2].name` — the 3rd element's name field (index 2).
8. normal structure variable — each element behaves like a regular struct variable.
9. `arr` (or `&arr[0]`) — `struct StructName *ptr = arr;`
10. for loop — loop with the dot operator prints each record's details.

■ II. Code Snippets

1. 100 students: `struct Student { int roll; char name[50]; }; struct Student s[100];` — loop `i=0..99` with `scanf("%d %s", &s[i].roll, s[i].name)`.
2. 5 employees: `struct Emp { int id; float salary; }; struct Emp e[5];` — input loop then display loop printing `e[i].id, e[i].salary`.
3. 10 students × 3 subjects: `struct Result { int marks[3]; }; struct Result r[10];` — `r[i].marks[j]` holds subject `j` of student `i`.
4. Shopping products: `struct Product { char name[50]; float price; }; struct Product p[50];`
5. Cricket team: `struct Player { char name[50]; int runs; }; struct Player team[11];`

■ III. Practice the Code

- `struct Car { char model[30]; int year; };`
- `struct Car c[3] = {{"Swift", 2020}, {"City", 2022}, {"Creta", 2023}};`
- `for (int i = 0; i < 3; i++) printf("%s (%d)\n", c[i].model, c[i].year);`